Hashing

1. Hashing

- Hashing is a technique used to store and retrieve data efficiently.
- A **hash function** converts a key into an index of a hash table.
- Average time complexity:
  - Search: O(1)
  - Insertion: O(1)
  - Deletion: O(1)
- Worst-case time complexity: O(n).

Hash Function

- A hash function maps a key to a valid table index.
- Common example:
  - `index = key % table_size`
- A good hash function should:
  - Be fast to calculate.
  - Distribute keys uniformly.
  - Minimize collisions.

SUNBEAM

---

2. Collision

- A collision occurs when two or more different keys generate the same hash index.
- Example:
  - Table size = 10
  - `h(25) = 25 % 10 = 5`
  - `h(35) = 35 % 10 = 5`
  - Therefore, 25 and 35 cause a collision.

- Collision handling is required to store colliding keys.

---

3. Collision Handling Techniques

Two major approaches are:

1. **Separate Chaining**
2. **Open Addressing**

3.1 Separate Chaining

- Each hash table index maintains a collection of elements.
- Usually, a linked list is used.
- Multiple keys with the same hash index are stored in the same chain.

Example:

```
Index 5 → 25 → 35 → 45
```

Advantages

- Simple to implement.
- Multiple elements can be stored at the same index.
- Load factor can be greater than 1.
- Deletion is relatively simple.

Disadvantages

- Extra memory is required for links/nodes.
- Long chains can make searching O(n).
- Cache performance is generally lower.

3.2 Open Addressing

SUNBEAM

- All elements are stored inside the hash table.
- When collision occurs, another empty position is searched.
- A probing technique is used.

Types:

1. Linear Probing
2. Quadratic Probing
3. Double Hashing

3.3 Linear Probing

- Searches the next available position sequentially.
- Probe sequence:
  - `h(k)`
  - `h(k) + 1`
  - `h(k) + 2`
  - ...
- Main problem: **Primary Clustering**.

Example:

```
5 → 6 → 7 → 8 → ...
```

3.4 Quadratic Probing

- Uses quadratic increments to find the next position.
- Probe sequence:
  - `h(k)`

SUNBEAM

- $h(k) + 1^2$
- $h(k) + 2^2$
- $h(k) + 3^2$
- ...

- Reduces primary clustering.
- Can suffer from **Secondary Clustering**.

3.5 Double Hashing

- Uses two hash functions.
- The second hash function determines the probing step.
- Formula:
  - $index = (h1(k) + i \times h2(k)) \% table_size$
- Provides better distribution.
- Reduces clustering.
- Requires an additional hash function.

---

4. Difference Between Collision Handling Techniques

Feature	Separate Chaining	Linear Probing	Quadratic Probing	Double Hashing
Storage	Outside table using chains	Inside table	Inside table	Inside table
Technique	Linked list/collection	Sequential probing	Quadratic probing	Second hash function
Extra memory	Required	No extra nodes	No extra nodes	No extra nodes

SUNBEAM

Feature	Separate Chaining	Linear Probing	Quadratic Probing	Double Hashing
Clustering	No probing clustering	Primary clustering	Secondary clustering	Very low
Load Factor	Can be > 1	Must be < 1	Must be < 1	Must be < 1
Deletion	Simple	More complicated	More complicated	More complicated

5. Linear vs Quadratic vs Double Hashing

Technique	Probe Sequence	Main Problem
Linear Probing	`h(k) + i`	Primary clustering
Quadratic Probing	`h(k) + i²`	Secondary clustering
Double Hashing	`h1(k) + i × h2(k)`	More computation

6. Rehashing

- Rehashing means creating a larger hash table and inserting all existing elements again.
- It is performed when the hash table becomes too full.
- A high **load factor** generally increases collisions.

Load Factor

- `Load Factor = Number of elements / Hash table size`
- Higher load factor:
    - More collisions.
    - More probing/chaining.
    - Lower performance.

Steps of Rehashing

SUNBEAM

1. Create a larger hash table.
2. Recalculate the hash index for every existing key.
3. Insert all keys into the new table.
4. Replace the old table.

Important Point

- Elements cannot simply be copied to the same indexes.
- The hash index may change because the table size has changed.

Example:

- Old table size = 5
- `h(12) = 12 % 5 = 2`

After rehashing:

- New table size = 7
- `h(12) = 12 % 7 = 5`
- Therefore, all elements must be reinserted.

Complexity of Rehashing

- Rehashing `n` elements takes **O(n)** time.
- Rehashing is an occasional operation.
- Hash table operations can still have **amortized O(1)** complexity.

---

7. Important MCQ Points

- Hashing provides average **O(1)** search, insertion and deletion.
- Collision occurs when different keys map to the same index.
- Separate chaining stores multiple elements in a chain.

SUNBEAM

- Open addressing stores all elements inside the hash table.
- Linear probing causes **primary clustering**.
- Quadratic probing reduces primary clustering.
- Quadratic probing can have **secondary clustering**.
- Double hashing uses **two hash functions**.
- Open addressing requires load factor **less than 1**.
- Rehashing means creating a larger table and **reinserting all existing elements**.
- Rehashing takes **O(n)** time for n existing elements.

SUNBEAM